

Task 1

TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    char filename[100];

    printf("Enter output file name: ");
    scanf("%s", filename);

    int saved_stdout = dup(STDOUT_FILENO);
    int saved_stderr = dup(STDERR_FILENO);

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("Cannot open output file");
        return 1;
    }

    /*
     Redirect stdout to file
    */
    dup2(fd, STDOUT_FILENO);
```

```

/*
    Redirect stderr to the same place as stdout
*/
dup2(STDOUT_FILENO, STDERR_FILENO);

close(fd);

printf("This message goes to stdout.\n");
fprintf(stderr, "This message goes to stderr.\n");

/*
    Restore streams
*/
dup2(saved_stdout, STDOUT_FILENO);
dup2(saved_stderr, STDERR_FILENO);

close(saved_stdout);
close(saved_stderr);

printf("Streams restored.\n");

return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 8$ ./task1
Enter output file name: output.txt
Streams restored.
geya@LenovoL480:~/OS/Skill/Week 8$ cat output.txt
This message goes to stdout.
This message goes to stderr.
geya@LenovoL480:~/OS/Skill/Week 8$ |

```

Task 2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == 0)
    {
        /*
         First command: ls
         Its output goes into the pipe.
        */

        dup2(pipefd[1], STDOUT_FILENO);
```

```
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("ls", "ls", NULL);

    perror("ls");
    return 1;
}

pid_t p2 = fork();

if (p2 == 0)
{
    /*
     * Second command: grep .c
     * Its input comes from the pipe.
     */

    int fd = open("output.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd == -1)
    {
        perror("output.txt");
    }
}
```

```

        return 1;
    }

    dup2(pipefd[0], STDIN_FILENO);
    dup2(fd, STDOUT_FILENO);

    close(fd);
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("grep");
    return 1;
}

close(pipefd[0]);
close(pipefd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Command completed.\n");
printf("Check output.txt for the result.\n");

return 0;

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 8$ ./task2
Command completed.
Check output.txt for the result.
geya@LenovoL480:~/OS/Skill/Week 8$ cat output.txt
task1.c
task2.c
geya@LenovoL480:~/OS/Skill/Week 8$ ls | grep .c > output.txt

```

Task 3

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char input[100];
    char *args[20];
    int count = 0;
    int background = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    /*
     Remove newline
    */
    input[strcspn(input, "\n")] = '\0';

    /*
     Check for &
    */
    int length = strlen(input);
```

```
if (length > 0 && input[length - 1] == '&')
{
    background = 1;
    input[length - 1] = '\\0';

    /* Remove trailing space */
    length = strlen(input);

    while (length > 0 && input[length - 1] == ' ')
    {
        input[length - 1] = '\\0';
        length--;
    }
}

/*
  Split command into arguments
*/
char *token = strtok(input, " ");
while (token != NULL && count < 19)
{
    args[count] = token;
    count++;
}
```

```

    token = strtok(NULL, " ");
}

args[count] = NULL;

if (count == 0)
{
    printf("No command entered.\n");
    return 1;
}

pid_t pid = fork();

if (pid == 0)
{
    /*
     * Child executes command
     */
    execvp(args[0], args);

    perror("Command failed");
    exit(1);
}
else if (pid > 0)
{

```

```

    if (background)
    {
        printf("Background process started. PID = %d\n", pid);
        printf("Prompt returned immediately.\n");
    }
    else
    {
        waitpid(pid, NULL, 0);
        printf("Foreground process completed.\n");
    }
}
else
{
    perror("fork");
}

return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 8$ ./task3
Enter command: sleep
sleep: missing operand
Try 'sleep --help' for more information.
Foreground process completed.
geya@LenovoL480:~/OS/Skill/Week 8$ ./task3
Enter command: sleep 10 &
Background process started. PID = 1137
Prompt returned immediately.
geya@LenovoL480:~/OS/Skill/Week 8$ ./task3
Enter command: sleep 3
Foreground process completed.
geya@LenovoL480:~/OS/Skill/Week 8$ |
```

Task 4

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int id;
    pid_t pid;
    char command[50];
    char state[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void show_jobs()
{
    int i;

    printf("\nJob Table\n");
    printf("-----\n");

    for (i = 0; i < job_count; i++)
```

```
{
    printf("[%d] PID=%d  Command=%s  State=%s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].command,
           jobs[i].state);
}

printf("-----\n");
}

void remove_job(int index)
{
    int i;

    for (i = index; i < job_count - 1; i++)
    {
        jobs[i] = jobs[i + 1];
    }

    job_count--;
}

void check_jobs()
{
    int i;
```

```

    int status;

    for (i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
        {
            printf("Job %d completed.\n", jobs[i].id);

            remove_job(i);
            i--;
        }
    }
}

int main()
{
    int choice;

    while (1)
    {

```

```
{
    check_jobs();

    printf("\n1. Start Background Job\n");
    printf("2. Show Jobs\n");
    printf("3. Exit\n");
    printf("Enter choice: ");

    scanf("%d", &choice);

    if (choice == 1)
    {
        if (job_count >= MAX_JOBS)
        {
            printf("Job table is full.\n");
            continue;
        }

        pid_t pid = fork();

        if (pid == 0)
        {
            execlp("sleep", "sleep", "5", NULL);

            perror("sleep");
            return 1;
        }
    }
}
```

```

    }
    else if (pid > 0)
    {
        jobs[job_count].id = job_count + 1;
        jobs[job_count].pid = pid;

        strcpy(jobs[job_count].command, "sleep 5");
        strcpy(jobs[job_count].state, "Running");

        job_count++;

        printf("Background job started. PID = %d\n", pid);
    }
    else
    {
        perror("fork");
    }
}
else if (choice == 2)
{
    show_jobs();
}
else if (choice == 3)
{
    break;
}

```

```

    else
    {
        printf("Invalid choice.\n");
    }
}

return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 8$ ./task4
```

1. Start Background Job
2. Show Jobs
3. Exit

Enter choice: 2

Job Table

1. Start Background Job
2. Show Jobs
3. Exit

Enter choice: 1

Background job started. PID = 1183

1. Start Background Job
2. Show Jobs
3. Exit

Enter choice: 3

```
geya@LenovoL480:~/OS/Skill/Week 8$ |
```